

AFRILANG Stdlib

std/c/cryptpbkdf

Bibliothèque AFRILANG `std/c/cryptpbkdf` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : pbkdLen, pbkdU

```
`import "std/c/cryptpbkdf" . `use cryptpbkdf`
```

- `pbkdLen(s text) ? number`
- `pbkdUpper(s text) ? text`
- `pbkdLower(s text) ? text`
- `pbkdTrim(s text) ? text`
- `pbkdSplit(s text, delim text) ? list text`
- `pbkdJoin(parts list text, delim text) ? text`
- `pbkdReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/c/cryptpbkdf`` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : `pbkdLen`, `pbkdU`

```
`import "std/c/cryptpbkdf"` . `use cryptpbkdf`  
- `pbkdLen(s text) ? number`  
- `pbkdUpper(s text) ? text`  
- `pbkdLower(s text) ? text`  
- `pbkdTrim(s text) ? text`  
- `pbkdSplit(s text, delim text) ? list text`  
- `pbkdJoin(parts list text, delim text) ? text`  
- `pbkdReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptpbkdf"  
use cryptpbkdf
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? pbkdLen(s text) ? number  
? pbkdUpper(s text) ? text  
? pbkdLower(s text) ? text  
? pbkdTrim(s text) ? text  
? pbkdSplit(s text, delim text) ? list  
? pbkdJoin(parts list text, delim text) ? text  
? pbkdReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/cryptpbkdf"  
use cryptpbkdf  
  
create result = pbkdLen(/* args */)   
say result  
  
create result = pbkdUpper(/* args */)   
say result  
  
create result = pbkdLower(/* args */)   
say result  
  
create result = pbkdTrim(/* args */)   
say result  
  
create result = pbkdSplit(/* args */)   
say result  
  
create result = pbkdJoin(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez ``import "std/c/cryptpbkdf"`` en tête de fichier.
- ? Lancez ``afirang check`` avant de livrer.
- ? Combinez avec les modules `stdlib` de la même catégorie.
- ? Voir `/stdlib/` pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module cryptpbkdf
function pbkdLen(s text) returns number
end
function pbkdUpper(s text) returns text
end
function pbkdLower(s text) returns text
end
function pbkdTrim(s text) returns text
end
function pbkdSplit(s text, delim text) returns list text
end
function pbkdJoin(parts list text, delim text) returns text
end
function pbkdReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library ``std/c/cryptpbkdf`` (tier complex, category complex). Exports 8 function(s): `pbkdLen`, `pbkdUpper`, `pbkdLow`

```
`import "std/c/cryptpbkdf"` . `use cryptpbkdf`  
- `pbkdLen(s text) ? number`  
- `pbkdUpper(s text) ? text`  
- `pbkdLower(s text) ? text`  
- `pbkdTrim(s text) ? text`  
- `pbkdSplit(s text, delim text) ? list text`  
- `pbkdJoin(parts list text, delim text) ? text`  
- `pbkdReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptpbkdf"  
use cryptpbkdf
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? pbkdLen(s text) ? number  
? pbkdUpper(s text) ? text  
? pbkdLower(s text) ? text  
? pbkdTrim(s text) ? text  
? pbkdSplit(s text, delim text) ? list  
? pbkdJoin(parts list text, delim text) ? text  
? pbkdReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/cryptpbkdf"  
use cryptpbkdf  
  
create result = pbkdLen(/* args */)   
say result  
  
create result = pbkdUpper(/* args */)   
say result  
  
create result = pbkdLower(/* args */)   
say result  
  
create result = pbkdTrim(/* args */)   
say result  
  
create result = pbkdSplit(/* args */)   
say result  
  
create result = pbkdJoin(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/cryptpbkdf"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module cryptpbkdf
function pbkdLen(s text) returns number
end
function pbkdUpper(s text) returns text
end
function pbkdLower(s text) returns text
end
function pbkdTrim(s text) returns text
end
function pbkdSplit(s text, delim text) returns list text
end
function pbkdJoin(parts list text, delim text) returns text
end
function pbkdReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```